SDD — Software Design Description

MANI

Arquitectura objetivo: SOA + API Gateway + enfoque políglota
Norma de calidad de referencia: ISO/IEC 25010:2023
Documento de datos asociado: ModeloDatos.md
Documento vivo: sin número de versión; la vigente es la de `main`. Es el **único lugar** donde viven los umbrales de calidad (§7 y §8).
Alcance: arquitectura de software, vistas C4, atributos de calidad, patrones, despliegue, ambientes y vista física.
Fuente de los diagramas C4: `workspace.dsl` — modelo Structurizr DSL, exportado a la estructura de ADR-0008 (`diagrams/c4/`, `diagrams/despliegue/` y `diagrams/flujos/dinamicos/`), con las vistas `panorama` (System Landscape), `contexto` (N1), `contenedores` (N2), `componentes-rules`, `componentes-dispatch`, `componentes-core` (N3), las dinámicas `dinamico-solicitud`, `dinamico-aceptacion`, `dinamico-cotizacion`, `dinamico-kyc`, `dinamico-mensajeria`, y la de despliegue `despliegue-prod`. Los diagramas de alto nivel (DHL) los referencia el `SAD.md`.

---

1. Propósito

Este Software Design Description (SDD) define la arquitectura de MANI desde la perspectiva de software. El documento cubre:

- arquitectura orientada a servicios (SOA);
- API Gateway como punto de entrada;
- enfoque políglota para servicios especializados;
- vistas C4 en los niveles de contexto, contenedores, componentes y código;
- decisiones y patrones arquitectónicos;
- atributos de calidad priorizados con base en ISO/IEC 25010:2023;
- escenarios medibles y umbrales de aceptación;
- vista de despliegue y ambientes;
- diseño físico del repositorio y de la infraestructura;
- estrategia de versionamiento, entrega y rollback.

> **Separación documental:** este SDD no redefine el modelo conceptual, lógico o físico de datos. La definición canónica de entidades, relaciones, DDL, diccionario de datos, Data Warehouse y flujo analítico se mantiene en ModeloDatos.md.

---

2. Alcance y principios

MANI es una plataforma multi-tenant para la creación y atención de solicitudes de servicio, cotización, selección y asignación de aliados, gestión KYC, disponibilidad, notificaciones y pagos.

Los principios que gobiernan el diseño son:

1. **La presentación no implementa reglas de negocio.**
2. **Los servicios son propietarios de su lógica de dominio.**
3. **El API Gateway es el punto de entrada de las APIs operacionales.**
4. **La persistencia se encapsula detrás de servicios y repositorios.**
5. **La tecnología puede variar por servicio, pero los contratos de integración son estables.**
6. **Los datos analíticos se separan del procesamiento transaccional.**

7. **Las decisiones arquitectónicas deben responder a atributos de calidad medibles.**

2.1 Decisiones explícitas de línea base

Para evitar ambigüedades entre ADR, diagramas y despliegue, se fijan las siguientes decisiones:

Tema	Decisión
Ambientes	**3 ambientes: DEV → QA → PROD**
Repositorios	**Multi-repo**
Persistencia	**Supabase como plataforma administrada**
Motor de base de datos	**PostgreSQL provisto por Supabase**
Aislamiento multi-tenant	**JWT + autorización en servicios + RLS en PostgreSQL/Supabase**
Orquestación	**Kubernetes**
Contenedores	**Docker/OCI**
CI/CD	**GitHub Actions**
Integración entre servicios, Sprint 4	**HTTP sincrónico por red interna, falla cerrado (`503`)**
Integración entre servicios, Sprint 5	**Bus de eventos con Apache Kafka**
Eventos de negocio	**Cuatro:** `SolicitudCreada`, `CotizacionAceptada` y `ServicioFinalizado` (Core), `AliadoAsignado` (Dispatch) — esquemas `v1` y versionado en `CONTRATOS_EVENTOS.md`
Publicación de eventos	**Transactional outbox**, obligatorio desde la primera implementación
Consumo de eventos	**Idempotente por `event_id`**
DDL ejecutable	**Serie única de migraciones propiedad de Core Service**

Supabase y PostgreSQL no son dos alternativas de persistencia. Supabase es la plataforma administrada utilizada por MANI y PostgreSQL es su motor relacional subyacente.

Kafka y Realtime no son dos alternativas de mensajería. Realtime notifica al usuario conectado (ADR-0017); Kafka transporta eventos de negocio entre servicios (ADR-0029). Son dos planos distintos, detallados en SAD §12.1 y §12.2. Mientras el bus no exista, la integración entre servicios es HTTP sincrónico con falla cerrado: si la dependencia no responde, el llamador devuelve `503` y **no** sustituye la respuesta por lógica local ni por un valor por omisión (ADR-0030 §2).

`ModeloDatos.md` no es el DDL ejecutable. Es la especificación normativa del modelo; lo que se aplica a la base es la serie única de migraciones propiedad de Core Service, con numeración incremental e inmutable. Rules y Dispatch no versionan DDL: abren PR de migración contra esa serie (ADR-0030 §1). La propiedad por servicio que enuncia §4.2 es **estado objetivo**, no descripción del presente.

2.2 Acceso del cliente a Supabase

El cliente Flutter alcanza Supabase por **exactamente dos caminos**, y **no existe ninguna excepción**:

Camino	Para qué	Por qué es legítimo
`Flutter → Supabase Auth`	iniciar sesión, registrar usuario y refrescar el JWT	la identidad la emite Supabase GoTrue y el tenant viaja como claim firmado (ADR-0018, ADR-0027)

Camino	Para qué	Por qué es legítimo
`Flutter ← Supabase Realtime`	recibir eventos de mensajería mientras el usuario está conectado	Realtime es un **transporte de eventos**: no lee tablas de negocio, no ejecuta reglas y no decide nada. El evento lo publica el Core después de persistir el mensaje

Todo lo demás —consultas, escrituras, archivos, procedimientos— va por `Flutter → API Gateway → servicio → Supabase`.

Queda prohibido y retirado, **sin excepción transitoria ni de disponibilidades**:

- `.from()` — acceso a tablas vía PostgREST;
- `.rpc()` — invocación de funciones almacenadas;
- `.storage.from()` — acceso directo a Storage. Los documentos KYC se suben por endpoint intermediario o URL prefirmada que emite el Core.

La consulta de disponibilidades **no** es una excepción: entra por el Gateway al Core como cualquier otra lectura de negocio.

> **Esta sección sustituye a la «excepción transitoria de disponibilidades».** Esa excepción quedaba abierta sin fecha de cierre y contradecía a ADR-0027, que está `Aceptado` y retira `.from()`, `.rpc()` y `.storage.from()` del cliente, y a ADR-0022, que saca la lógica de negocio del cliente y de las funciones PL/pgSQL.

---

3. Estilo arquitectónico

3.1 SOA

MANI se define como una **arquitectura orientada a servicios (SOA)**. Las capacidades del negocio se organizan en servicios con responsabilidades explícitas, interfaces contractuales y bajo acoplamiento.

Capacidades principales:

Servicio / capacidad	Responsabilidad
Reglas por Tenant	ranking, validación tarifaria, políticas KYC y configuración por tenant
Despacho y Asignación	solicitudes, candidatos, aceptación/rechazo, exclusiones, concurrencia y auditoría
Core de Negocio	usuarios, tenants, aliados, KYC, catálogos, notificaciones y reportes operativos
Disponibilidades	categorías, zonas, horarios y disponibilidad operacional
Integraciones	FCM/APNs, pasarela de pagos, observabilidad y servicios externos

La orientación a servicios permite que la lógica del dominio no quede acoplada a Flutter ni a la base de datos.

3.2 API Gateway

NGINX funciona como frontera de entrada para las APIs.

Responsabilidades:

- terminación HTTPS;
- enrutamiento;
- autenticación y propagación de identidad;
- rate limiting;
- balanceo;

- control de CORS;
- trazabilidad mediante correlation ID;
- políticas básicas de seguridad.

El Gateway **no debe contener reglas de negocio**.

3.3 Enfoque políglota

La solución utiliza diferentes tecnologías de implementación según el servicio:

Componente	Tecnología
Aplicación web/móvil	Flutter / Dart
Gateway	NGINX
Servicio de reglas	Java
Despacho y asignación	.NET
Servicios restantes	Node.js
Persistencia	Supabase (PostgreSQL como motor)
Contenedores	Docker
Orquestación	Kubernetes
CI/CD	GitHub Actions

El reparto no es por gusto ni por coleccionar frameworks: cada stack tiene una responsabilidad delimitada y los diagramas C4 la muestran agrupando los contenedores por bloque.

```
JAVA            .NET            NODE
decide          asigna          opera
  ↓               ↓               ↓
reglas de       despacho y      servicios funcionales
negocio         asignación      de la plataforma
```

- **MANI-Rules-Service — decide.** Un único servicio: evalúa si una condición de negocio se cumple. Ranking, elegibilidad, requisitos KYC, tarifarios y reglas por tenant. No es un backend general.
- **MANI-Dispatch-Service — asigna.** Un único servicio: selecciona aliados válidos, asigna, controla estados y exclusiones. Consume Rules antes de asignar; no reimplementa las reglas.
- **MANI-Core-Service — opera.** El núcleo funcional: usuarios y tenants, identidad, aliados, solicitudes, cotizaciones, documentos y multimedia, notificaciones, reportes, disponibilidad y ubicación.

El enfoque políglota no implica libertad tecnológica irrestricta. Cada tecnología debe justificar su existencia, mantener contratos estables y cumplir los mismos criterios de seguridad, observabilidad, pruebas y despliegue.

---

4. Vistas C4

Las vistas de esta sección se generan desde `workspace.dsl`, que es su fuente. Cada subsección indica la vista que le corresponde y mantiene en texto la estructura y las responsabilidades, para que el documento se lea sin renderizar.

El modelo cubre las cuatro vistas que Structurizr sí puede describir, y ninguna queda solo en prosa:

Vista	Pregunta que responde	Dónde
Panorama — System Landscape	qué sistemas existen en el mapa y a cuáles toca cada actor	§4.5
Estática — contexto, contenedores, componentes	qué existe y cómo se relaciona	§4.1 a §4.3

Vista	Pregunta que responde	Dónde
Dinámica	en qué orden ocurre cada flujo crítico	§4.6
Despliegue	dónde se ejecuta cada contenedor, de dónde sale y quién lo vigila	§9

El Nivel 4 (Code) se mantiene en §4.4: Structurizr describe contenedores y componentes, no clases.

4.0 Inventario de diagramas

Las doce vistas del proyecto están definidas en `workspace.dsl` y exportadas a `diagrams/c4/`, `diagrams/despliegue/` y `diagrams/flujos/dinamicos/` según ADR-0008. El DSL es la fuente: un diagrama no se edita en la imagen, se cambia en el modelo y se vuelve a exportar. Si una vista no aparece en esta tabla, no es oficial.

#	Vista en el DSL	Tipo	Qué muestra	Imagen	Sección
1	`panorama`	System Landscape	El mapa completo: MANI como único sistema propio, sus actores —incluido el visitante sin sesión de la landing— y las plataformas externas con las que convive. Es la vista para explicar el proyecto a quien no lo conoce	`panorama.png`	§4.5
2	`contexto`	C4 Nivel 1	MANI como una sola caja, sus actores y los sistemas externos. A diferencia del panorama, pone el foco en un único sistema	`contexto.png`	§4.1
3	`contenedores`	C4 Nivel 2	Las unidades desplegables —landing comercial, cliente, Gateway y los tres servicios —, los almacenes de Supabase, las integraciones y la analítica	`contenedores.png`	§4.2
4	`componentes-rules`	C4 Nivel 3	Dentro del Rules Service: controlador, servicio de aplicación, las estrategias por tipo de regla, el puerto y su adaptador	`componentes-rules.png`	§4.3.1
5	`componentes-dispatch`	C4 Nivel 3	Dentro del Dispatch Service: selector de candidatos, coordinador de asignación, **concurrency guard** y auditoría	`componentes-dispatch.png`	§4.3.2
6	`componentes-core`	C4 Nivel 3	Dentro del Core Service: identidad y tenants, KYC, catálogo, cobertura y disponibilidad, ciclo del servicio, comunicaciones, reportes y adaptadores externos	`componentes-core.png`	§4.3.3
7	`dinamico-solicitud`	Dinámica	El orden en que se crea una solicitud: el despacho pide elegibilidad al Core y el orden del listado al Rules Service	`dinamico-solicitud.png`	§4.6.1
8	`dinamico-aceptacion`	Dinámica	Dos aliados aceptan a la vez y la actualización condicional atómica deja pasar solo al primero; el segundo recibe `409`	`dinamico-aceptacion.png`	§4.6.2
9	`dinamico-cotizacion`	Dinámica	El Core persiste la cotización y el Rules Service la contrasta contra el tarifario de referencia	`dinamico-cotizacion.png`	§4.6.3
10	`dinamico-kyc`	Dinámica	Los documentos del aliado llegan al bucket privado y el administrador del tenant aprueba o rechaza	`dinamico-kyc.png`	§4.6.4
11	`dinamico-mensajeria`	Dinámica	El mensaje se persiste primero, Realtime lo transporta al participante conectado y el push cubre al desconectado	`dinamico-mensajeria.png`	§4.6.5
12	`despliegue-prod`	Despliegue	Dónde se ejecuta cada contenedor en producción, de dónde sale su imagen y quién lo vigila	`despliegue-prod.png`	§9

Dos aclaraciones sobre el alcance de la tabla:

- **El Nivel 4 (Code) no está en el DSL** y por eso no tiene fila: Structurizr describe contenedores y componentes, no clases. Vive como texto en §4.4.
- **Los diagramas de alto nivel no son vistas del modelo.** `DHL.png`, `Infra.png` y `TechRadar.png` viven en `diagrams/c4/contexto/`, `diagrams/despliegue/` y `diagrams/TechRadar.png`, no se generan desde el DSL y los referencian el `SAD.md` y el `TECH_RADAR.md`.

Cada subsección de abajo repite en texto la estructura de su vista, para que el documento se pueda leer sin abrir las imágenes.

> ⚠️ **Las imágenes exportadas están desactualizadas y cuatro están recortadas**
>
> **Las imágenes exportadas no corresponden al `workspace.dsl` vigente.** Son exportaciones de una versión anterior del modelo. Mientras no se regeneren, **manda el DSL y manda el texto de este documento**, nunca la imagen.
>
> Diferencias comprobadas entre lo que muestran las imágenes y lo que dice el modelo:
>
> | La imagen muestra | El modelo dice |
> |---|---|
> | `Availability Service` como contenedor propio | no existe: la disponibilidad es un dominio del Core Service |
> | grupos `MANI-Core-Node`, `MANI-Rules-Java`, `MANI-Dispatch-DotNet` | `MANI-Core-Service`, `MANI-Rules-Service`, `MANI-Dispatch-Service` |
> | `Core Services`, en plural | `Core Service` |
> | `dinamico-aceptacion` cita ADR-0021 | ADR-0021 se retiró el 2026-10-23; la decisión vigente es ADR-0016, que lo absorbe |
>
> **Además, una exportación sigue cortada por el borde derecho**, con nodos partidos a media palabra: `dinamico-aceptacion`, que mide exactamente 4096 px de ancho.
>
> `contenedores`, `componentes-rules` y `componentes-dispatch` ya se regeneraron con el límite elevado y están completas (7820, 6580 y 6580 px de ancho respectivamente).
>
> La causa no es el modelo: es el límite por defecto de PlantUML, que es el renderizador de estas vistas. `PLANTUML_LIMIT_SIZE` vale 4096 px y **recorta en lugar de escalar**. Al regenerar hay que subirlo:
>
> ```
> bash export PLANTUML_LIMIT_SIZE=16384
> ```
>
> Las demás vistas están completas y se leen bien; el problema de recorte afecta solo a esa.

Cómo leer las imágenes

Todas las figuras son clicables: abren la imagen a resolución completa. Hace falta usarlo, porque varias vistas son muy anchas y al escalarlas al ancho de la página las etiquetas quedan ilegibles. Estas son las que casi siempre hay que abrir:

Vista	Resolución	Proporción
`dinamico-aceptacion`	4096 × 660	6,2 : 1
`dinamico-cotizacion`	3499 × 672	5,2 : 1
`dinamico-kyc`	2969 × 634	4,7 : 1
`dinamico-solicitud`	2705 × 672	4,0 : 1
`dinamico-mensajeria`	2578 × 677	3,8 : 1
`componentes-rules`	6580 × 2108	3,1 : 1
`componentes-dispatch`	6580 × 2327	2,8 : 1
`contenedores`	7820 × 5302	1,5 : 1

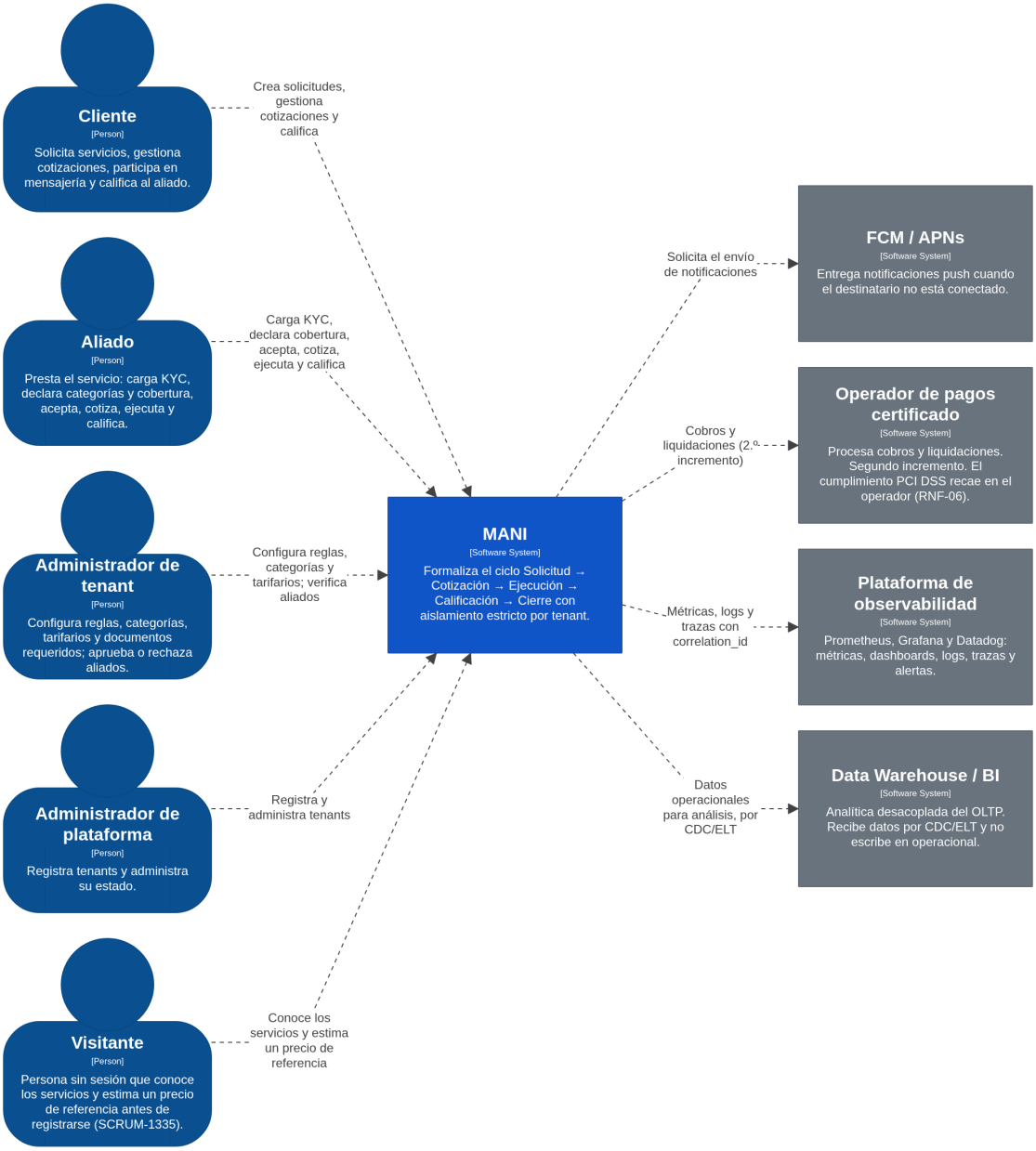
Vista	Resolución	Proporción
`componentes-core`	5000 × 3828	1,3 : 1

Las vistas dinámicas son anchas por naturaleza: una secuencia crece hacia los lados con cada paso. Por eso **cada figura va acompañada del flujo en texto**, que es normativo: el documento se puede leer y revisar sin abrir ninguna imagen, y en una revisión de PR el texto es lo que se compara.

Si una imagen contradice al texto, manda el texto: la imagen es una exportación del modelo y puede estar sin regenerar.

4.1 Nivel 1 — System Context

Objetivo: mostrar MANI como un sistema y sus relaciones con personas y sistemas externos.



System Context View: MANI

C4 Nivel 1 — MANI, sus actores y los sistemas externos. Fuente: SAD §5, SDD §4.1.

lunes, 19 de octubre de 2026, 4:03 a. m. hora estándar de Colombia

> **Figura 1 — Contexto del sistema (C4 Nivel 1).** Generada desde la vista `contexto` de `workspace.dsl`; imagen en `diagrams/c4/contexto/contexto.png`.

```
Cliente             → crea solicitudes y aprueba cotizaciones
Aliado              → cotiza, acepta y ejecuta servicios; gestiona KYC        → [ MANI ]
Administrador tenant → administra usuarios, aliados, tenant y KYC
Visitante (sin sesión) → conoce los servicios y estima un precio en la landing

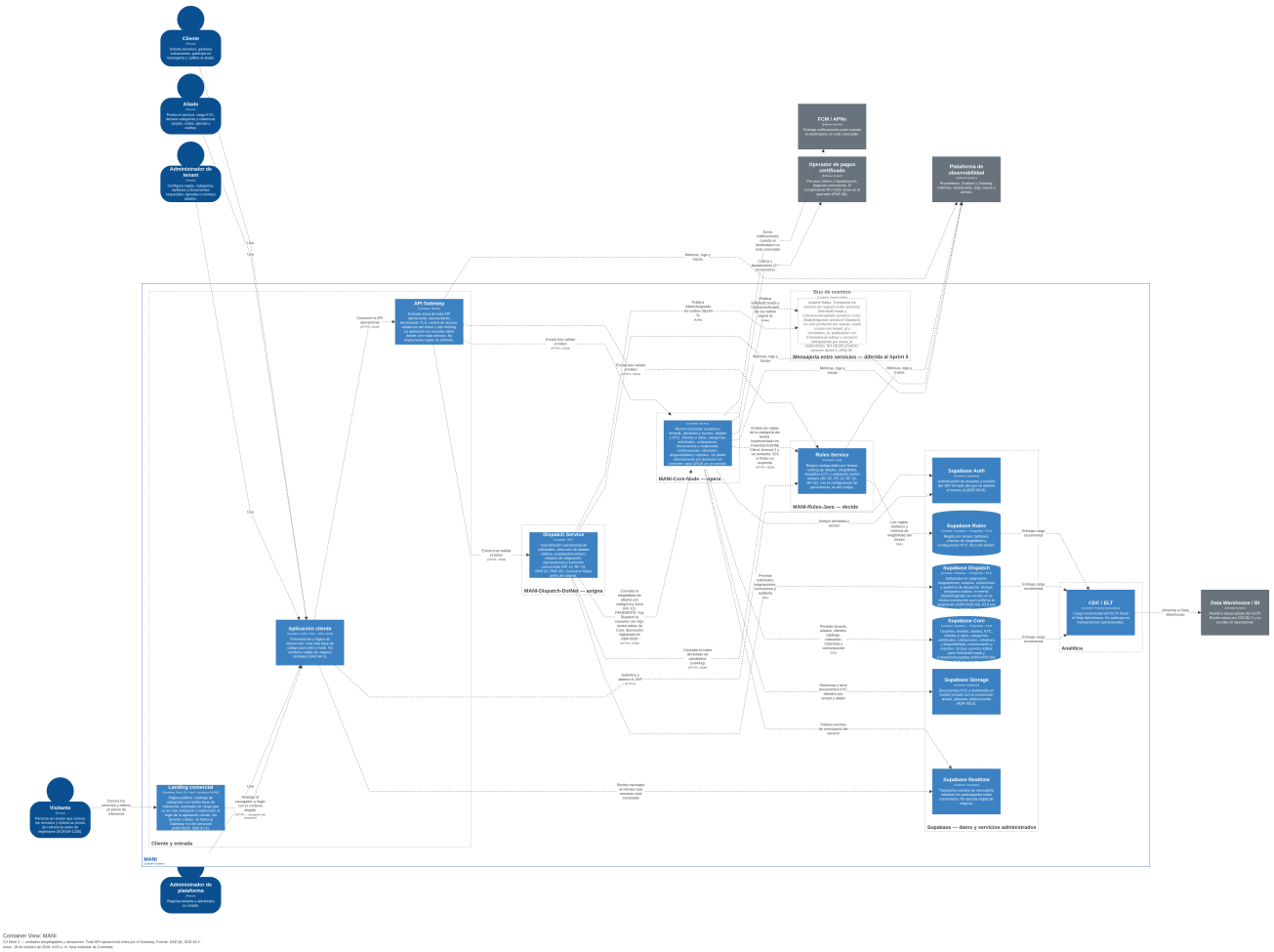
[ MANI ] → FCM / APNs                    notificaciones
         → Operador de pagos             API de pagos
         → Plataforma de observabilidad  métricas, logs y trazas
         → Data Warehouse                datos operacionales para análisis
```

Responsabilidades de frontera

- **MANI** ejecuta el flujo transaccional.
- **FCM/APNs** entrega notificaciones push.
- **Pasarela de pagos** procesa pagos; MANI conserva referencias y estados, no datos sensibles de tarjeta.
- **Observabilidad** recibe métricas, logs y trazas.
- **Data Warehouse** recibe datos mediante procesos desacoplados del flujo transaccional.

4.2 Nivel 2 — Containers

Objetivo: mostrar las unidades desplegables y almacenes principales.



Container View: MANI

> **Figura 2 — Contenedores (C4 Nivel 2).** Generada desde la vista `contenedores` de `workspace.dsl`; imagen en `diagrams/c4/contenedores/contenedores.png`.

```
Visitante → Landing comercial (React + Vite) ──redirección del navegador a /login──┐
                                                                                    ↓
Usuarios → Flutter Web / Mobile → (HTTPS) → NGINX API Gateway
                      enrutamiento · control de acceso · token · rate limiting
                                                ↓
   ┌── decide ──────────┬── asigna ──────────┬── opera ──────────────────┐
   Rules Service        Dispatch Service     Core Service
   (Java)               (.NET)               (Node.js)
        ↓                    ↓                    ↓
   Supabase Rules       Supabase Dispatch    Supabase Core
        └──────────── PostgreSQL + RLS por tenant ──────────┘
```

```
                                                    ↓
  Core → FCM/APNs · Operador de pagos · Storage · Realtime
  Gateway y servicios → telemetría → Prometheus / Grafana / Datadog
  PostgreSQL → carga incremental → CDC/ELT → Data Warehouse → BI
```

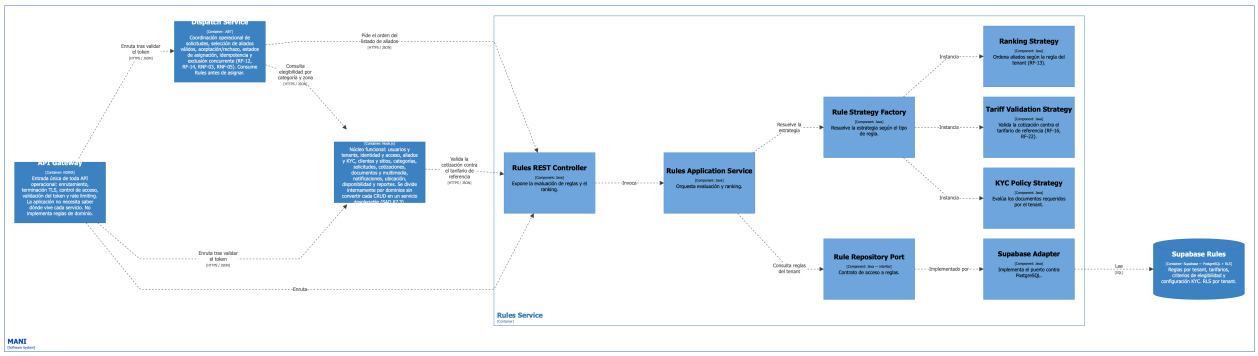
Dispatch consulta a Core la elegibilidad por categoría y zona, y a Rules el orden del listado; Core consulta a Rules para validar la cotización contra el tarifario. Son llamadas HTTPS entre servicios, no accesos cruzados a datos.

Reglas de dependencia

- Flutter consume contratos expuestos por el Gateway.
- La landing comercial no consume ningún contrato: no llama al Gateway, a los servicios ni a Supabase. Solo redirige el navegador al login de Flutter Web (SAD §7.4, ADR-0028).
- Cada servicio es dueño de su almacén: Rules de `Supabase Rules`, Dispatch de `Supabase Dispatch` y Core de `Supabase Core`, que incluye el dominio de disponibilidad.
- Los servicios no acceden directamente a tablas propiedad de otro dominio.
- Las integraciones externas se encapsulan mediante adaptadores.
- El Data Warehouse no participa en transacciones operacionales.
- El detalle de esquemas, entidades y modelo dimensional se encuentra en `ModeloDatos.md`.

4.3 Nivel 3 — Components

4.3.1 Rules Service — Java



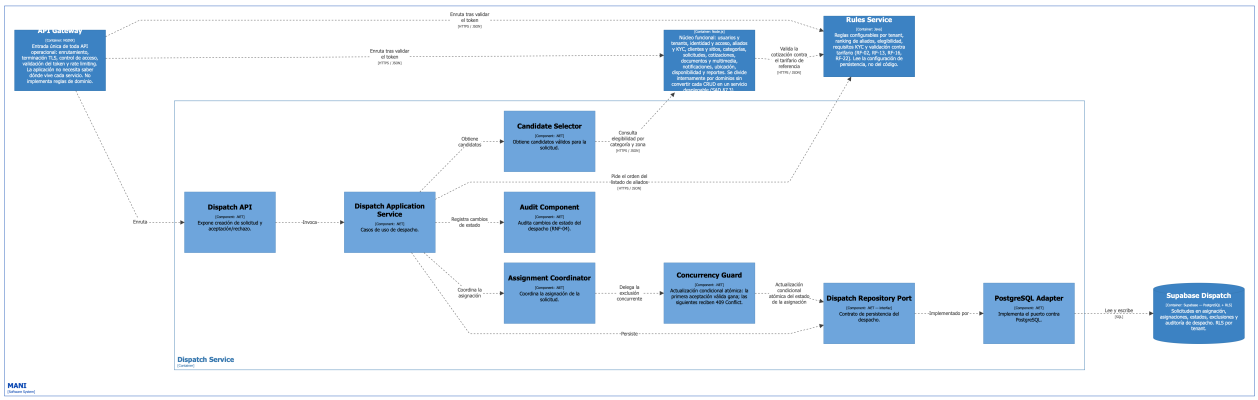
> **Figura 3 — Rules Service, Java (C4 Nivel 3).** Generada desde la vista `componentes-rules` de `workspace.dsl`; imagen en `diagrams/c4/componentes/componentes-rules.png`.

```
Rules REST Controller
  → Rules Application Service
       → Rule Strategy Factory → Ranking Strategy | Tariff Validation Strategy | KYC Policy Strategy
       → Rule Repository Port  → Supabase Adapter (PostgreSQL) → Reglas DB
```

Responsabilidades:

- seleccionar reglas por tenant;
- calcular ranking de aliados;
- validar cotización contra tarifario;
- validar requisitos KYC;
- desacoplar lógica de reglas de la persistencia.

4.3.2 Dispatch Service — .NET



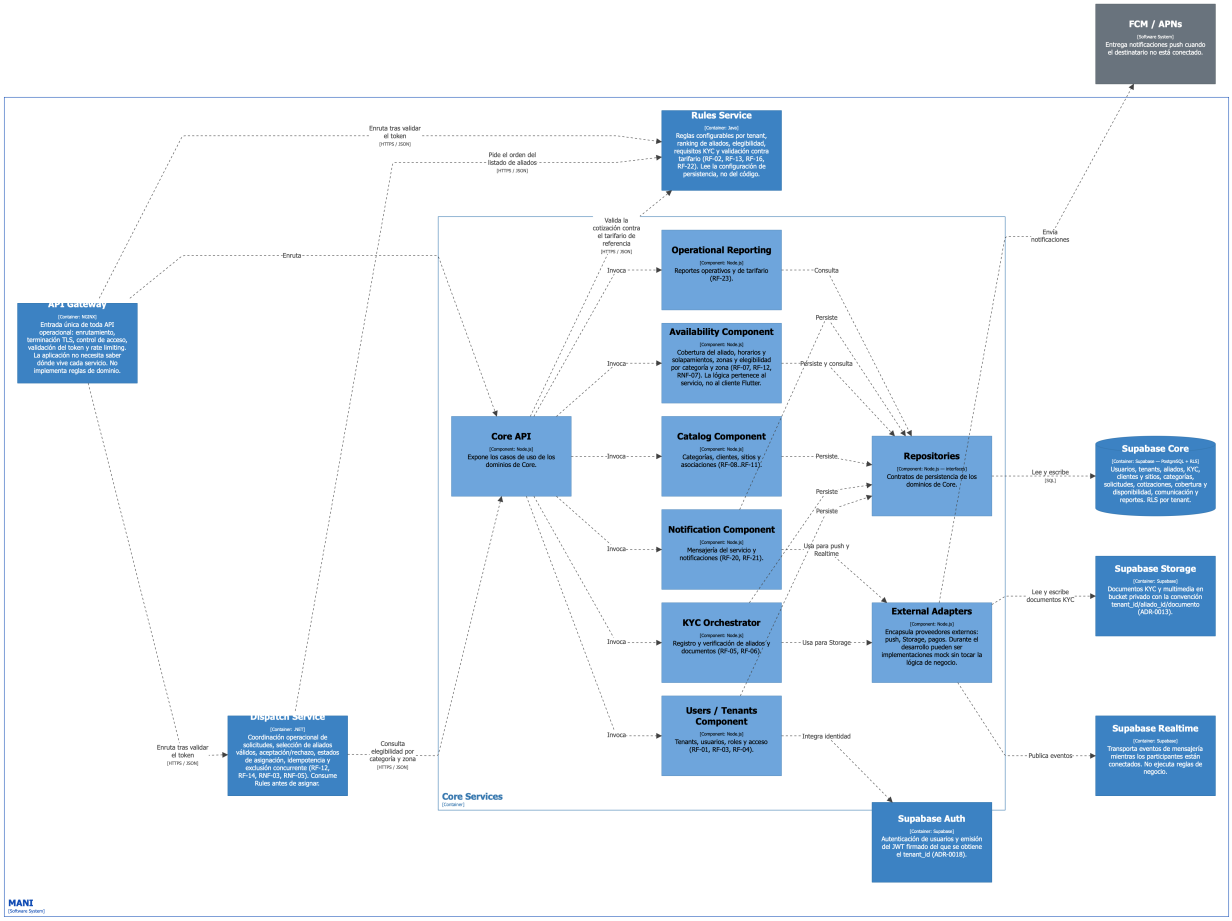
> **Figura 4 — Dispatch Service, .NET (C4 Nivel 3).** Generada desde la vista `componentes-dispatch` de `workspace.dsl`; imagen en `diagrams/c4/componentes/componentes-dispatch.png`.

```
Dispatch API
  → Dispatch Application Service
       → Candidate Selector
       → Assignment Coordinator → Concurrency Guard
       → Audit Component
       → Dispatch Repository Port → PostgreSQL Adapter → Dispatch DB
```

Responsabilidades:

- crear solicitudes;
- obtener candidatos válidos;
- coordinar asignación;
- evitar doble aceptación;
- responder conflicto `409` cuando la asignación ya fue tomada;
- auditar cambios de estado.

4.3.3 Core Service — Node.js



> **Figura 5 — Core Service, Node.js (C4 Nivel 3).** Generada desde la vista `componentes-core` de `workspace.dsl`; imagen en `diagrams/c4/componentes/componentes-core.png`.

```
Core API
  → Users/Tenants Component ─┐
  → KYC Orchestrator ────────┤
  → Catalog Component ───────┤
  → Availability Component ──┼→ Repositories → Core DB
  → Notification Component ──┤
  → Operational Reporting ───┘

KYC Orchestrator y Notification Component → External Adapters → Storage · Realtime · FCM/APNs
```

Availability Component concentra cobertura del aliado, horarios y solapamientos, zonas y elegibilidad por categoría y zona (RF-07, RF-12, RNF-07). Es el componente que Despacho consulta en caliente antes de conformar el listado de candidatos. La lógica pertenece al servicio, no al cliente Flutter: esa es la condición para cerrar la excepción transitoria de §2.1.

La disponibilidad **no es un servicio desplegable aparte**. Vive en el bloque Node como un dominio más de Core, igual que el catálogo o la comunicación: separarla en su propio despliegue sería convertir un dominio en un servicio sin ganar nada a cambio (SAD, riesgo *Servicios excesivamente pequeños*).

4.4 Nivel 4 — Code

El nivel 4 expresa la estructura interna de código. No pretende congelar clases concretas para siempre; define las dependencias que deben preservarse.

Ejemplo: Rules Service

```
RulesController
  +evaluateRule(request)
  +rankAllies(request)
      ↓
RulesApplicationService
  +evaluate(context)
  +rank(context)
      ↓                              ↓
RuleStrategyFactory             RuleRepository  «interfaz»
  +resolve(ruleType)              +findByTenant(tenantId)
      ↓                              ↑
RuleStrategy  «interfaz»        PostgresRuleRepository
  +supports(ruleType)             +findByTenant(tenantId)
  +evaluate(context)
      ↑
  RankingStrategy · TariffStrategy · KycStrategy   (implementaciones)
```

Ejemplo: Dispatch Service

```
DispatchController
  → CreateRequestUseCase → RequestRepository  «interfaz»
  → AssignAllyUseCase    → CandidateSelector
                         → ConcurrencyGuard
                         → AssignmentRepository  «interfaz»
                         → AuditPublisher
```

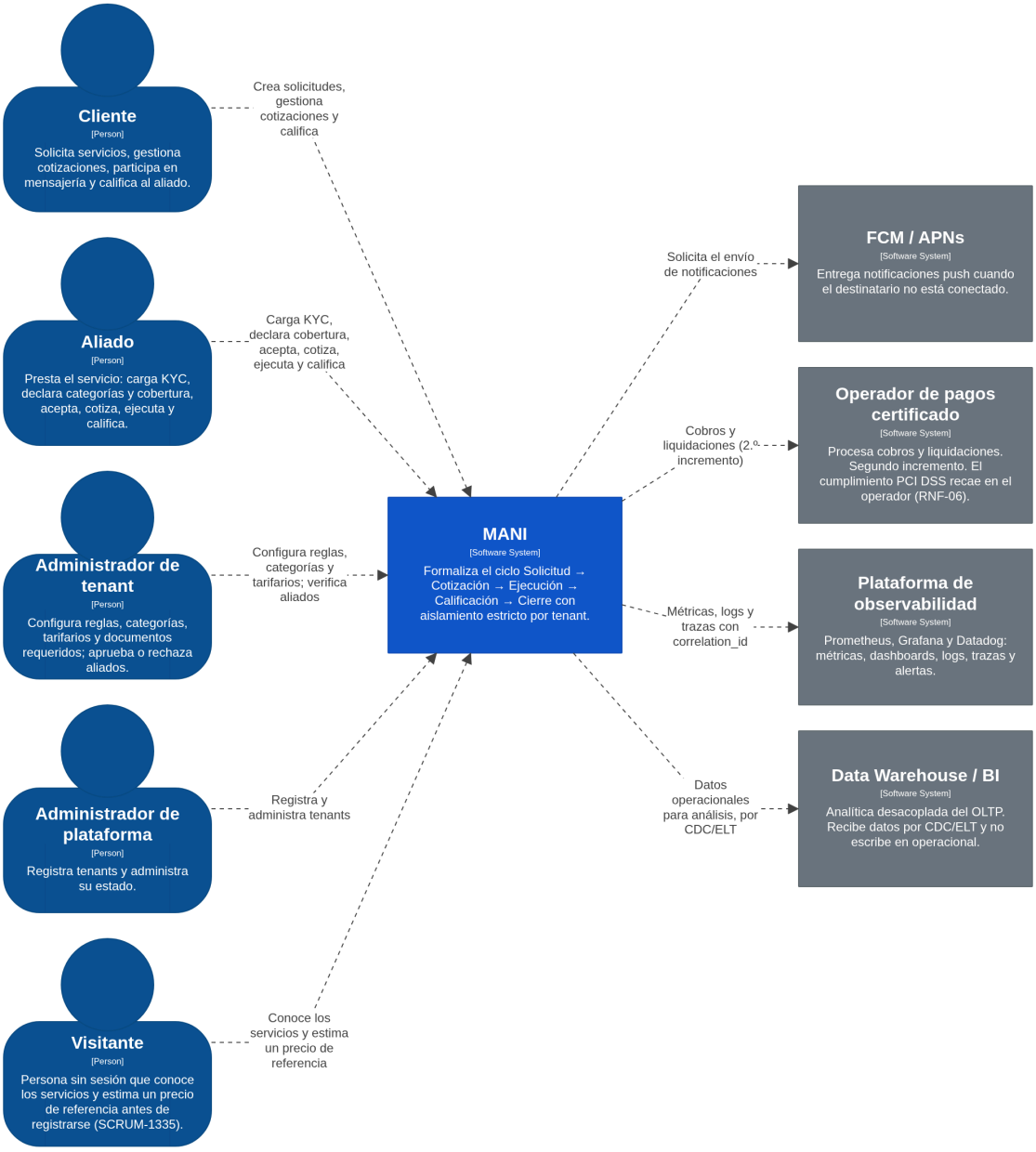
Regla de dependencia de código

La dependencia debe apuntar desde adaptadores hacia contratos internos, no desde el dominio hacia frameworks externos:

```
Controller / Adapter
        ↓
Application / Use Cases
        ↓
Domain
        ↑
Ports / Interfaces
        ↑
DB / External Adapters
```

4.5 Panorama de sistemas — System Landscape

Objetivo: mostrar el mapa de sistemas alrededor de MANI. Es una pregunta distinta de la del Nivel 1: el contexto mira hacia fuera *desde* MANI, el panorama mira el conjunto y deja ver que MANI es el único sistema propio y que todo lo demás es proveedor o plataforma de destino.



System Landscape View

Panorama de sistemas: MANI, los actores que lo usan y las plataformas externas de las que depende. Fuente: SAD §5, SRS §2.3.

lunes, 19 de octubre de 2026, 4:03 a. m. hora estándar de Colombia

> **Figura 6 — Panorama de sistemas (System Landscape).** Generada desde la vista `panorama` de `workspace.dsl`; imagen en `diagrams/c4/contexto/panorama.png`.

```
Cliente · Aliado · Administrador de tenant · Administrador de plataforma
                              ↓
                        [ MANI ]  ← único sistema propio
                              ↓
   FCM / APNs · Operador de pagos · Observabilidad · Data Warehouse / BI
                    (proveedores y plataformas de destino)
```

Consecuencia de diseño: cada sistema externo entra al modelo por un adaptador, nunca por acoplamiento directo de un servicio al proveedor (§6 y §14).

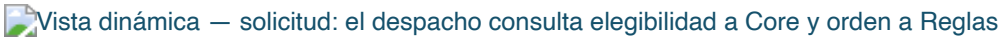
4.6 Vistas dinámicas

Objetivo: mostrar el orden de los flujos que la estructura estática no explica. Cada paso de estas vistas corresponde a una relación ya declarada en el modelo: una vista dinámica no inventa colaboraciones que la estructura no permita.

> Los mismos flujos, en **PlantUML suelto y con las ramas de error** (`alt`, `409 Conflict`, destinatario desconectado), están en `SECUENCIAS.md`. Esa notación admite lo que el modelo C4 no expresa: dos actores compitiendo y los caminos alternativos.

Vista	Flujo	Requisitos
`dinamico-solicitud`	solicitud y conformación del listado de aliados	RF-12, RF-13
`dinamico-aceptacion`	exclusión concurrente en la aceptación	RF-14, RNF-03, RNF-05
`dinamico-cotizacion`	cotización y alerta contra el tarifario	RF-15, RF-16, RF-22
`dinamico-kyc`	carga y verificación de documentos KYC	RF-05, RF-06
`dinamico-mensajeria`	mensajería con notificación de respaldo	RF-20, RF-21

4.6.1 Solicitud y listado de aliados — `dinamico-solicitud`


Vista dinámica — solicitud: el despacho consulta elegibilidad a Core y orden a Reglas

> **Figura 7 — Solicitud y listado de aliados.** Generada desde la vista `dinamico-solicitud`; imagen en `diagrams/flujos/dinamicos/dinamico-solicitud.png`.

```
1. Cliente            → Flutter          registra la solicitud
2. Flutter            → API Gateway      POST con el JWT del tenant
3. API Gateway        → Dispatch         enruta tras validar el token
4. Dispatch           → Core             aliados elegibles por categoría y zona (RF-12)
5. Dispatch           → Rules            orden del listado según la regla del tenant (RF-13)
6. Dispatch           → PostgreSQL       persiste solicitud y candidatos notificados
```

Quien orquesta es el despacho: pregunta elegibilidad al dominio de disponibilidad en Core y el orden a Reglas. Ni el cliente Flutter ni el Gateway deciden nada de esto.

4.6.2 Aceptación concurrente — `dinamico-aceptacion`


Vista dinámica — aceptación concurrente: la actualización condicional atómica deja pasar la primera aceptación

> **Figura 8 — Aceptación concurrente.** Generada desde la vista `dinamico-aceptacion`; imagen en `diagrams/flujos/dinamicos/dinamico-aceptacion.png`.

Vista a nivel de componentes del Servicio de Despacho. Dos aliados aceptan la misma solicitud al mismo tiempo:

```
1. API Gateway          → Dispatch API             aceptación con clave de idempotencia (RNF-03)
2. Dispatch API         → Application Service      invoca el caso de uso
3. Application Service  → Assignment Coordinator   coordina la asignación
4. Coordinator          → Concurrency Guard        delega la exclusión concurrente
5. Concurrency Guard    → Repository Port          actualización condicional: solo si sigue libre
6. Repository Port      → PostgreSQL Adapter       implementación
7. Adapter              → PostgreSQL               UPDATE condicionado al estado previo
8. Application Service  → Audit Component          audita el cambio de estado (RNF-04)
```

La primera aceptación afecta una fila y gana. La segunda no afecta ninguna y recibe `409 Conflict`. La garantía está en la actualización condicional, no en la aplicación.

4.6.3 Cotización y tarifario — `dinamico-cotizacion`

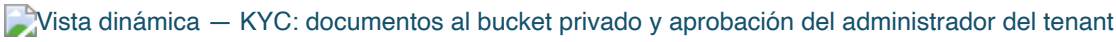


> **Figura 9 — Cotización y tarifario.** Generada desde la vista `dinamico-cotizacion`; imagen en `diagrams/flujos/dinamicos/dinamico-cotizacion.png`.

```
1. Aliado        → Flutter        cotización separando mano de obra y materiales (RF-15)
2. Flutter       → API Gateway    envía la cotización
3. API Gateway   → Core           enruta tras validar el token
4. Core          → Rules          valida el valor contra el rango del tarifario (RF-16)
5. Rules         → PostgreSQL     lee mínimo, típico y máximo del tenant (RF-22)
6. Core          → PostgreSQL     persiste la cotización con el resultado
```

Core es dueño de la cotización (§7.3 del SAD) y Reglas es dueño del tarifario (§7.1 del SAD): la validación es una llamada entre servicios, no lógica duplicada en Core.

4.6.4 KYC del aliado — `dinamico-kyc`

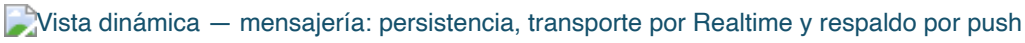


> **Figura 10 — KYC del aliado.** Generada desde la vista `dinamico-kyc`; imagen en `diagrams/flujos/dinamicos/dinamico-kyc.png`.

```
1. Aliado                → Flutter        carga los documentos requeridos por el tenant
2. Flutter               → API Gateway    envía los documentos
3. API Gateway           → Core           enruta tras validar el token
4. Core                  → Storage        bucket privado tenant_id/aliado_id/documento
5. Core                  → PostgreSQL     registra el documento; aliado en verificación
6. Administrador tenant  → Flutter        aprueba o rechaza al aliado (RF-06)
```

La aprobación es una decisión humana del tenant, no un efecto automático de la carga.

4.6.5 Mensajería y notificación — `dinamico-mensajeria`



> **Figura 11 — Mensajería y notificación.** Generada desde la vista `dinamico-mensajeria`; imagen en `diagrams/flujos/dinamicos/dinamico-mensajeria.png`.

```
1. Cliente   → Flutter        escribe un mensaje del servicio
2. Flutter   → API Gateway    envía el mensaje
3. Gateway   → Core           enruta tras validar el token
4. Core      → PostgreSQL     persiste el mensaje
5. Core      → Realtime       publica el evento de mensajería
6. Flutter   → Realtime       el destinatario conectado lo recibe casi en tiempo real
7. Core      → FCM / APNs     si no está conectado, lo notifica por push (RF-21)
```

El mensaje se persiste antes de transportarse: la conversación no vive en el canal de tiempo real. Realtime transporta, no decide.

---

5. Patrones arquitectónicos

Patrón	Aplicación en MANI	Justificación
SOA	capacidades separadas como servicios	bajo acoplamiento y evolución independiente

Patrón	Aplicación en MANI	Justificación
API Gateway	NGINX	punto de entrada, políticas transversales y ocultamiento de topología interna
Layered Architecture interna	API → aplicación → dominio → infraestructura	evita mezclar HTTP, reglas y persistencia
Repository / Ports & Adapters	acceso a Supabase mediante interfaces sobre PostgreSQL	desacopla lógica de negocio del proveedor de datos
Database ownership por dominio	esquemas/repositorios separados	limita dependencias y cambios cruzados
Event-driven para efectos secundarios	notificaciones, auditoría, analítica	reduce cadenas síncronas
Externalized Configuration	secretos y configuración fuera del código	portabilidad y seguridad
Observability	métricas, logs, trazas y correlation ID	diagnóstico de sistemas distribuidos

6. Patrones de diseño

Patrón	Uso
Strategy	reglas variables por tenant
Factory	resolución de la estrategia correspondiente
Repository	abstracción de persistencia
Adapter	FCM/APNs, pagos, KYC y otros proveedores
Facade / Application Service	coordinación de casos de uso
Observer / Publish-Subscribe	notificaciones, auditoría y eventos analíticos
Circuit Breaker	protección frente a fallas de servicios externos
Retry con backoff	reintentos controlados en operaciones idempotentes
Outbox	publicación confiable de eventos tras una transacción
Idempotency Key	evitar pagos, asignaciones o comandos duplicados

7. Atributos de calidad — ISO/IEC 25010:2023

ISO/IEC 25010:2023 define un modelo de calidad de producto con nueve características. MANI las utiliza como marco, pero **no todas reciben la misma prioridad**.

Escala:

- **P1 — Crítico:** condiciona decisiones estructurales y aceptación del sistema.
- **P2 — Alto:** debe cumplirse antes de producción, pero no domina todas las decisiones.
- **P3 — Controlado:** se verifica, aunque su impacto arquitectónico es menor para el alcance actual.

Los umbrales siguientes son **objetivos de aceptación inicial del MVP**. Deben recalibrarse con pruebas de carga y datos reales.

7.1 Priorización general

Característica ISO/IEC 25010:2023	Prioridad	Razón
Seguridad	P1	multi-tenant, KYC, identidad, pagos y datos sensibles

Característica ISO/IEC 25010:2023	Prioridad	Razón
Fiabilidad	P1	solicitudes, despacho y asignación requieren continuidad y consistencia
Eficiencia de desempeño	P1	ranking, disponibilidad y despacho son sensibles a latencia
Mantenibilidad	P1	arquitectura políglota y múltiples servicios elevan el costo de cambio
Flexibilidad	P1	los servicios deben escalar, desplegarse y adaptarse por separado
Compatibilidad	P2	integración entre Java, .NET, Node.js, Flutter y terceros
Adecuación funcional	P2	exactitud de reglas, asignaciones y estados
Capacidad de interacción	P2	cliente, aliado y administrador necesitan flujos claros
Protección (Safety)	P3	no es un sistema safety-critical, pero debe evitar operaciones irreversibles inseguras

---

7.2 Seguridad — P1

Subcaracterística	Prioridad	Umbral / criterio de aceptación
Confidencialidad	P1	100% de endpoints privados requieren autenticación; TLS 1.2+; secretos fuera del código
Integridad	P1	100% de operaciones críticas protegidas por validación de autorización y constraints transaccionales
Autenticidad	P1	tokens firmados y validados en 100% de llamadas protegidas
Responsabilidad / accountability	P1	100% de cambios críticos generan actor, timestamp y correlation ID
No repudio	P2	pagos, aceptación y cambios críticos conservan evidencia auditable
Resistencia	P1	0 vulnerabilidades Critical/High conocidas abiertas en release de producción; rate limiting en Gateway

Controles arquitectónicos: JWT/OIDC, API Gateway, RBAC, aislamiento por tenant, RLS cuando aplique, auditoría, SAST con SonarQube, DAST con OWASP ZAP, gestión de secretos y mínimos privilegios.

7.3 Fiabilidad — P1

Subcaracterística	Prioridad	Umbral / criterio
Ausencia de fallos	P1	tasa de error 5xx < 1% en ventana de 15 min bajo carga nominal
Disponibilidad	P1	≥ 99.9% mensual para APIs críticas
Tolerancia a fallos	P1	falla de proveedor de notificaciones no cancela una solicitud o asignación confirmada
Recuperabilidad	P1	RTO ≤ 30 min; RPO ≤ 5 min para datos operacionales críticos

Controles: réplicas, health checks, restart automático, circuit breaker, retry controlado, backups, restauración probada y procesamiento asíncrono de efectos secundarios.

7.4 Eficiencia de desempeño — P1

Subcaracterística	Prioridad	Umbral / criterio
Comportamiento temporal	P1	p95 ≤ 500 ms para lecturas internas simples y p95 ≤ 800 ms para comandos críticos, excluyendo tiempo de terceros
Utilización de recursos	P2	utilización sostenida objetivo < 70% CPU por pod antes de escalar

Subcaracterística	Prioridad	Umbral / criterio
Capacidad	P1	soportar inicialmente 300 sesiones concurrentes y 50 req/s sostenidos sin incumplir p95

Controles: índices, consultas acotadas, paginación, caché selectiva, HPA, pooling, pruebas de carga y métricas.

7.5 Mantenibilidad — P1

Subcaracterística	Prioridad	Umbral / criterio
Modularidad	P1	ningún servicio accede directamente a tablas de otro dominio
Reusabilidad	P2	contratos y adaptadores comunes reutilizables sin duplicar reglas de dominio
Analizabilidad	P1	100% de requests distribuidos con correlation ID; logs estructurados
Capacidad para ser modificado	P1	cambios internos compatibles no exigen modificar consumidores externos
Capacidad para ser probado	P1	cobertura de código nuevo ≥ 80%; casos de uso críticos ≥ 90%; contract tests obligatorios

Quality Gate: 0 issues Blocker/Critical en código nuevo; duplicación de código nuevo < 3%; pipeline debe fallar si el Quality Gate no se cumple.

7.6 Flexibilidad — P1

Subcaracterística	Prioridad	Umbral / criterio
Adaptabilidad	P1	misma imagen desplegable en DEV/QA/PROD mediante configuración externa
Escalabilidad	P1	servicios críticos escalables horizontalmente de 2 a 6 réplicas sin cambio de código
Instalabilidad	P1	despliegue automatizado por ambiente ≤ 10 min en condiciones normales
Reemplazabilidad	P2	proveedores externos encapsulados por Adapter; cambio de proveedor sin alterar dominio

---

7.7 Compatibilidad — P2

Subcaracterística	Prioridad	Umbral / criterio
Coexistencia	P2	servicios respetan límites configurados de CPU/memoria y no degradan otros workloads
Interoperabilidad	P1 dentro de la característica	100% de APIs documentadas con OpenAPI; contract tests verdes antes de promoción

---

7.8 Adecuación funcional — P2

Subcaracterística	Prioridad	Umbral / criterio
Completitud funcional	P2	100% de historias críticas trazadas a casos de prueba
Corrección funcional	P1 dentro de la característica	100% de escenarios críticos de ranking, tarifa, asignación y conflicto pasan pruebas
Pertinencia funcional	P2	cada endpoint productivo corresponde a un caso de uso identificado

---

7.9 Capacidad de interacción — P2

Subcaracterística	Prioridad	Umbral / criterio
Reconocibilidad	P2	usuario identifica acción principal de cada flujo sin documentación externa
Aprendizabilidad	P3	usuarios de prueba completan flujos base con ≤ 1 asistencia
Operabilidad	P2	≥ 90% de éxito en pruebas de tareas críticas
Protección contra errores de usuario	P1 dentro de la característica	acciones irreversibles requieren validación/confirmación y errores accionables
Involucración	P3	consistencia visual definida por sistema de diseño
Inclusividad	P2	criterios de accesibilidad definidos para web/móvil
Asistencia al usuario	P3	errores y estados vacíos incluyen orientación
Auto-descriptividad	P2	formularios y estados críticos explican el siguiente paso

7.10 Protección / Safety — P3

MANI no controla maquinaria ni procesos donde un fallo de software implique directamente riesgo de vida. Aun así:

Subcaracterística	Prioridad	Umbral / criterio
Restricción operativa	P2	estados inválidos de solicitud/asignación deben rechazarse
Identificación de riesgos	P3	operaciones sensibles identificadas en análisis de riesgo
Protección ante fallos	P2	fallas de terceros no deben dejar transacciones críticas en estado ambiguo
Advertencia de peligro	P3	mensajes explícitos antes de cancelaciones/reversiones críticas
Integración segura	P2	integración externa requiere timeout, validación de respuesta y circuit breaker cuando aplique

8. Escenarios de calidad priorizados

Esta sección operacionaliza la priorización definida en la sección 7 mediante **un escenario de calidad representativo por cada característica de ISO/IEC 25010:2023**. Cada escenario identifica la fuente del estímulo, el estímulo, el entorno de ejecución, el artefacto afectado, la respuesta esperada, el umbral de aceptación y el mecanismo mediante el cual QA debe verificarlo.

> **Estado del método de verificación — corte 6 de octubre de 2026.** El método descrito en la columna "¿Cómo lo mide QA?" es el acordado para cada escenario, no una declaración de que el escenario ya se verificó. Su ejecución depende de las herramientas de QA-01 a QA-05, ninguna entregada a la fecha de corte: Testcontainers (QA-01, SCRUM-1118), Playwright (QA-02, SCRUM-1119), la colección de contrato con Newman (QA-03, SCRUM-1120), k6 (QA-04, SCRUM-1121) y OWASP ZAP (QA-05, SCRUM-1122). Maestro, que aparece en QAS-08, no tiene ticket. Las colecciones de Newman y los scripts de k6 que existen hoy en `MANI-Frontend/qa/` son del Sprint 2 y recorren el camino directo a Supabase: sirven como línea base y para verificar RLS como defensa adicional, no como contrato Gateway–Core ni como escenario de carga de estos QAS. Los gates de ADR-0005 que dependen de SonarQube no son medibles para `MANI-Core-Service` ni `MANI-API-Gateway` hasta CFG-40 (SCRUM-1115) y CFG-41 (SCRUM-1116). Prometheus y Grafana, que aparecen en QAS-05, no se han verificado como aprovisionados; mientras no lo estén, aplica la misma salvedad. Mientras una herramienta no esté entregada, el escenario que depende de ella se reporta como **no medible**, nunca como cumplido. El detalle por nivel y su estado vive en `wiki/01-producto/TD.md` §2.1.

ID	Característica ISO/IEC 25010:2023	Subcaracterística seleccionada	Prioridad	Fuente	Estímulo	Entorno	Artefacto	Respuesta esperada	Umbral / criterio de aceptación	¿Cómo lo mide QA?
QAS-01	**Seguridad**	**Confidencialidad**	P1	Usuario autenticado de un tenant	Intenta consultar o modificar información perteneciente a otro tenant	QA o PROD en operación normal	API Gateway, servicios de dominio y PostgreSQL/RLS	El servicio rechaza la operación tomando el tenant del claim del JWT firmado y no de un dato enviado por el cliente, no expone información del tenant destino y registra el evento cuando corresponda; RLS actúa como capa adicional de defensa en profundidad	**100% de accesos cross-tenant rechazados**	QA ejecuta con Postman/Newman, a través del Gateway, los seis casos cross-tenant de las Políticas DevOps §13.4 (lectura de otro tenant, listado de recursos ajenos, escritura en otro tenant, borrado ajeno, token expirado o alterado, y documentos KYC de otro tenant o aliado), con tokens y recursos de al menos dos tenants; verifica rechazo de acceso, ausencia de datos ajenos, evidencia de auditoría y que el rechazo ocurre en la autorización del servicio y no solo en RLS. Evidencia: reporte de Newman con versión del servicio y fecha
QAS-02	**Fiabilidad**	**Tolerancia a fallos**	P1	Proveedor externo de notificaciones	FCM/APNs deja de responder después de confirmarse una operación de negocio	QA con dependencia externa degradada o indisponible, simulada mediante un adaptador de prueba	Core Service, Notification Component y adaptador externo	La operación principal permanece confirmada y la notificación queda disponible para reintento controlado	**0 operaciones confirmadas revertidas únicamente por fallo de notificación**	QA simula timeout, error y caída del proveedor con un adaptador de prueba; tras cada fallo verifica el estado final de la operación, la persistencia del evento de notificación, los logs con correlation ID y la ejecución del mecanismo de retry. Evidencia: registro de cada simulación con el estado final de la operación
QAS-03	**Eficiencia de desempeño**	**Comportamiento temporal**	P1	Usuario o servicio consumidor	Consulta disponibilidad de aliados por categoría y zona	QA bajo carga nominal: 300 sesiones concurrentes y 50 req/s sostenidos (sección 7.4)	API Gateway, Core Service (dominio de disponibilidad), persistencia y componentes involucrados en la consulta	El sistema retorna candidatos elegibles dentro del tiempo objetivo	**p95 ≤ 500 ms** para la consulta bajo carga nominal	QA ejecuta un escenario k6 propio de la consulta por categoría y zona con la carga nominal sostenida, registra los tiempos de respuesta del endpoint y calcula el percentil 95; el escenario se aprueba si el p95 permanece dentro del umbral. El resultado declara el camino medido (Gateway → Core Service, dominio de disponibilidad, o, mientras rija la excepción de la sección 2.1, acceso directo a Supabase), la carga, el ambiente y la fecha, y no se extrapola desde otro escenario

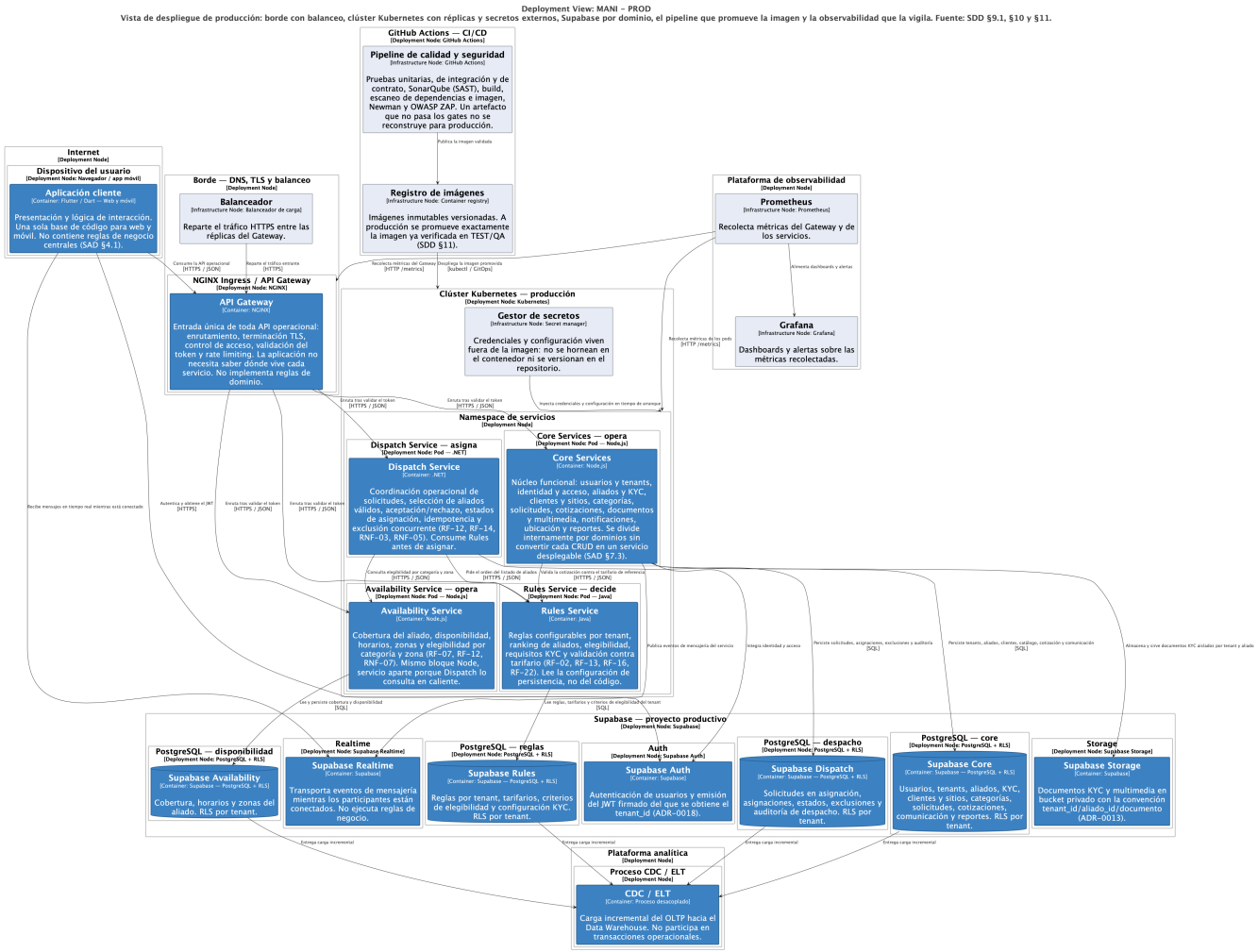
ID	Característica ISO/IEC 25010:2023	Subcaracterística seleccionada	Prioridad	Fuente	Estímulo	Entorno	Artefacto	Respuesta esperada	Umbral / criterio de aceptación	¿Cómo lo mide QA?
QAS-04	**Mantenibilidad**	**Capacidad para ser modificado**	P1	Equipo de desarrollo	Modifica una regla de ranking o comportamiento configurable de un tenant manteniendo el contrato externo	DEV y QA	Rules Service (estrategias de ranking, tarifa y KYC por tenant), su configuración y los contratos de integración	El cambio se implementa solo en Rules Service y su configuración, sin exigir modificaciones en consumidores externos compatibles	**0 cambios obligatorios en Flutter, Gateway, Dispatch ni en otros servicios consumidores de Rules para un cambio interno compatible**	QA ejecuta pruebas de regresión, integración y contract tests (Newman); toma el PR del cambio y verifica por su diff que solo toca Rules Service y su configuración, y que los consumidores existentes continúan operando sin modificaciones derivadas del cambio. Evidencia: enlace al PR y reporte de contract tests
QAS-05	**Flexibilidad**	**Escalabilidad**	P1	Incremento de demanda del sistema	La carga de un servicio crítico supera la capacidad objetivo de las réplicas actuales	Kubernetes en QA bajo carga controlada	Deployment, HPA y servicio crítico contenerizado (Rules, Dispatch o Core)	La plataforma incrementa horizontalmente la capacidad del servicio cuando la utilización sostenida de CPU por pod supera el 70% (sección 7.4), sin modificar código ni reconstruir el artefacto	**Escalamiento de 2 a 6 réplicas sin cambio de código**	QA genera carga progresiva con k6 y monitorea réplicas, CPU, memoria, latencia, errores y continuidad del servicio mediante Kubernetes y la plataforma de observabilidad (Prometheus/Grafana); verifica que el servicio escala de 2 a 6 réplicas con la misma imagen. Evidencia: gráficas de réplicas y CPU junto con la configuración del HPA
QAS-06	**Compatibilidad**	**Interoperabilidad**	P2	Servicio interno o consumidor autorizado	Consume una API publicada por un servicio implementado en otra tecnología	QA de integración	APIs REST de los servicios publicadas por el API Gateway y sus contratos OpenAPI	Productor y consumidor intercambian información respetando el contrato publicado	**100% de APIs documentadas con OpenAPI y contract tests críticos aprobados antes de promoción**	QA contrasta las APIs publicadas por el Gateway con los contratos OpenAPI (cada API publicada debe tener el suyo), ejecuta contract tests y pruebas de integración/Newman entre servicios de distinta tecnología; valida códigos HTTP, payloads, tipos de datos, campos obligatorios y compatibilidad del contrato. Evidencia: tabla API ↔ contrato y reporte de contract tests

ID	Característica ISO/IEC 25010:2023	Subcaracterística seleccionada	Prioridad	Fuente	Estímulo	Entorno	Artefacto	Respuesta esperada	Umbral / criterio de aceptación	¿Cómo lo mide QA?
QAS-07	**Adecuación funcional**	**Corrección funcional**	P2	Usuarios o servicios que ejecutan reglas y operaciones críticas	Ejecutan escenarios críticos de ranking, tarifa, asignación o conflicto	QA con datos de prueba controlados	Rules Service, Dispatch Service y servicios involucrados	El sistema produce el resultado funcional definido para cada regla, asignación y transición de estado	**100% de escenarios críticos de ranking, tarifa, asignación y conflicto aprobados**	QA mantiene casos de prueba trazados a los requisitos críticos y verifica resultados esperados, estados persistidos, códigos de respuesta y reglas de negocio; para asignación y conflicto incluye la aceptación concurrente de una misma solicitud sobre Dispatch con k6 (una aceptación confirmada con 200, las demás con 409, 0 dobles asignaciones y reintentos idempotentes). Evidencia: matriz requisito ↔ caso y reporte de ejecución
QAS-08	**Capacidad de interacción**	**Protección contra errores de usuario**	P2	Usuario autorizado	Intenta ejecutar una acción irreversible o envía información inválida en un flujo crítico	Aplicación web/móvil en operación normal	Flutter (presentación) y servicio de dominio asociado al caso de uso, detrás del Gateway	El sistema previene la ejecución accidental mediante validación o confirmación y devuelve errores accionables; el servicio valida también las entradas y no depende solo del cliente	**100% de acciones irreversibles definidas requieren validación o confirmación previa**	QA recorre los flujos críticos de pantalla en Flutter con Maestro y prueba entradas inválidas, cancelaciones y acciones irreversibles; verifica validaciones, confirmaciones, mensajes y conservación del estado previo cuando la operación es rechazada, y comprueba con Newman que el servicio rechaza con error accionable las entradas inválidas enviadas sin pasar por la interfaz. Evidencia: reporte de Maestro y de Newman
QAS-09	**Protección / Safety**	**Protección ante fallos**	P3	Servicio o integración externa	Se produce una falla durante una operación que modifica información crítica	QA con falla controlada de una dependencia	Servicio de negocio (por ejemplo Core con el adaptador de Storage en la carga de documentos KYC), adaptador externo y persistencia	El sistema evita estados parciales o ambiguos y conserva un estado consistente o recuperable	**0 transacciones críticas en estado ambiguo por fallas de terceros**	QA aplica fault injection, timeouts o respuestas inválidas con adaptadores de prueba durante operaciones críticas y valida estados persistidos, logs, eventos y posibilidad de recuperación sin inconsistencias. A diferencia de QAS-02, la falla ocurre durante la operación y no después de confirmarla. Evidencia: registro de cada inyección con el estado persistido resultante

Los escenarios anteriores complementan los umbrales específicos definidos en las subsecciones 7.2 a 7.10. Cuando exista diferencia entre un escenario de esta tabla y un criterio más restrictivo definido para una subcaracterística en la sección 7, **prevalece el criterio más restrictivo**.

9. Vista de despliegue

9.1 Producción



> **Figura 12 — Despliegue de producción.** Generada desde la vista `despliegue-prod` de `workspace.dsl`; imagen en `diagrams/despliegue/despliegue-prod.png`.

```
Web / Mobile Users → DNS + TLS + balanceador → NGINX Ingress / API Gateway (2 réplicas)
                                                ↓
Clúster Kubernetes — producción (proveedor y topología pendientes: INFRA-01, INFRA-02)
  Namespace de servicios
    Rules — decide (Java) · Dispatch — asigna (.NET) · Core — opera (Node.js)
    2 réplicas mínimo cada uno, HPA hasta 6
  Gestor de secretos → inyecta credenciales y configuración fuera de la imagen
                                                ↓
Supabase — proyecto productivo
  PostgreSQL por dominio: reglas · despacho · core (RLS por tenant)
  Auth · Storage (bucket privado) · Realtime
                                                ↓
Core → FCM/APNs + Operador de pagos
PostgreSQL → CDC/ELT → Data Warehouse

GitHub Actions → pipeline de calidad y seguridad → registro de imágenes → clúster
Prometheus → recolecta métricas del Gateway y de los pods → Grafana (dashboards y alertas)
```

La vista incluye deliberadamente **de dónde sale** lo que corre —el pipeline promueve exactamente la misma imagen verificada— y **quién lo vigila**. Ambas cosas son parte del despliegue y no se deducen de la estructura lógica.

Reglas físicas

- contenedores inmutables;
- al menos dos réplicas para servicios críticos en producción;
- readiness y liveness probes;
- requests/limits de CPU y memoria;
- HPA para servicios sensibles a carga;
- secretos suministrados desde gestor seguro;
- bases productivas no accesibles desde Internet pública salvo controles explícitos;
- acceso administrativo con privilegio mínimo.

Los demás ambientes no se modelan como vistas aparte: Local, DEV y QA comparten esta topología y solo cambian escalado, secretos y datos (§10). Un diagrama por ambiente repetiría la misma información sin añadir ninguna decisión arquitectónica.

---

10. Ambientes

MANI adopta **tres ambientes de despliegue**. No se define un ambiente STAGING independiente.

Ambiente	Objetivo	Infraestructura	Datos
Local	desarrollo individual	Docker Compose + Flutter + servicios + Supabase local	datos sintéticos
DEV	integración continua temprana	Kubernetes con aislamiento lógico del ambiente	datos sintéticos
QA	pruebas funcionales, integración, seguridad y validación previa a producción	Kubernetes con aislamiento lógico + instancia/base separada	dataset controlado y anonimizado
PROD	operación real	Kubernetes productivo + Supabase productivo	datos reales

Reglas

- el flujo oficial es **DEV → QA → PROD**;
- no existe STAGING como ambiente obligatorio;
- no compartir bases, secretos ni credenciales entre ambientes;
- no copiar datos KYC o datos personales reales a DEV o QA;
- configuración y secretos son externos a las imágenes;
- la misma imagen de contenedor validada se promueve entre QA y PROD;
- únicamente cambian parámetros externos, secretos y escalado;
- Kubernetes sigue siendo el orquestador; la separación física por clúster o namespace puede ajustarse sin cambiar esta decisión.

---

11. Flujo CI/CD y promoción

```
Commit / Pull Request
  → Unit + Integration + Contract Tests
  → SonarQube (SAST)
  → Build Images
```

```
  → Dependency / Image Scan
  → Deploy QA
  → Newman + OWASP ZAP
  → Promoción del mismo artefacto validado a PROD
  → Verificación post-despliegue
```

Un artefacto que no pasa los gates de calidad **no se reconstruye para producción**; se promueve exactamente la misma imagen verificada.

---

12. Versionamiento de despliegue

12.1 Servicios

Se utiliza **Semantic Versioning**:

```
MAJOR.MINOR.PATCH
```

Ejemplo:

```
rules-service:2.3.1
dispatch-service:1.8.0
core-service:1.5.4
```

- **MAJOR:** cambio incompatible de contrato.
- **MINOR:** funcionalidad compatible.
- **PATCH:** corrección compatible.

La imagen final debe quedar además fijada por digest o commit SHA.

12.2 API

Los cambios incompatibles se versionan en la ruta o contrato:

```
/api/v1/...
/api/v2/...
```

Una versión MAJOR anterior se mantiene durante una ventana de transición definida.

12.3 Base de datos

Las migraciones son incrementales e inmutables:

```
V001__initial_schema.sql
V002__add_assignment_status.sql
V003__add_rule_priority.sql
```

No se modifica una migración ya ejecutada en ambientes compartidos; se crea una nueva.

12.4 Releases

Ejemplo:

```
v1.4.0-dev.3
v1.4.0-rc.1
v1.4.0
```

La etiqueta final representa el release productivo.

12.5 Rollback

- rollback de aplicación mediante imagen previa;
- migraciones destructivas requieren estrategia expand/contract;
- cambios de esquema incompatibles no se despliegan en el mismo paso que la eliminación de compatibilidad;
- objetivo de rollback operacional: ≤ 5 min.

---

13. Vista física de repositorios

MANI mantiene una estrategia **multi-repositorio**. La arquitectura distribuida no se representa como un monorepo.

```
GitHub / Organización MANI
├── MANI-Frontend
│   └── aplicación web/móvil Flutter
├── MANI-API-Gateway
│   └── configuración NGINX / API Gateway
├── MANI-Rules-Service
│   └── servicio de reglas por tenant
├── MANI-Dispatch-Service
│   └── servicio de despacho y asignación
├── MANI-Core-Service
│   ├── tenants, identidad, clientes y sitios, aliados y KYC, catálogo
│   ├── cobertura, disponibilidad y elegibilidad
│   ├── cotización, ejecución y calificación
│   └── comunicaciones
├── MANI-Landing
│   └── página comercial pública (React + Vite); solo redirige al login del cliente
└── MANI-Docs
    ├── product/        SRS y backlog
    ├── architecture/   SAD.md, SDD.md, ModeloDatos.md, workspace.dsl, TECH_RADAR.md
    ├── adr/
    ├── governance/     gobierno, políticas DevOps e infraestructura
    ├── diagrams/       HLD (DHL) y LLD (vistas exportadas)
    └── wiki/           navegación y reglas derivadas
```

Responsabilidades

- cada repositorio desplegable mantiene su propio ciclo de build, test y versionamiento;
- `MANI-API-Gateway` es además la raíz de composición del despliegue: guarda `database/`, `scripts/`, `supabase/` y el Compose por ambiente (ADR-0004, enmienda del 2026-10-07);
- **no existe `MANI-Infra`**: DevOps lo borró el 2026-10-06. Dónde vivirán los manifests de orquestación y la configuración de observabilidad queda abierto con INFRA-01 e INFRA-02;
- **no existe `MANI-Availability`**: la cobertura y la disponibilidad son un dominio del `MANI-Core-Service` (ADR-0028);
- `MANI-Landing` tiene pipeline e imagen propios, no accede a datos ni al Gateway y su única relación con el resto es la redirección al login de `MANI-Frontend` (ADR-0028);
- `MANI-Docs` contiene la documentación técnica oficial y no se despliega;

- GitHub Actions puede reutilizar workflows compartidos, pero cada artefacto se construye y versiona de manera independiente;
- la promoción entre ambientes preserva el mismo artefacto validado;
- la estrategia multi-repo es una decisión explícita del proyecto (ADR-0004) y los nombres los fija ADR-0028.

> **Pendiente de decisión —** `MANI-Availability`**.** Desde §4.3.3 la disponibilidad es un dominio interno de Core, no un servicio desplegable, así que ese repositorio ya no corresponde a ningún artefacto desplegable propio. La lista de repositorios la fija ADR-0004, de modo que retirarlo o reconvertirlo en librería compartida exige un ADR nuevo que lo supersede, no una edición de este documento. Mientras tanto el repositorio queda declarado aquí tal como lo definió esa decisión.

---

14. Dependencias permitidas y prohibidas

Permitidas

```
Flutter → API Gateway
API Gateway → Servicios
Servicio → Repositorio propio
Servicio → Adapter → Proveedor externo
OLTP → CDC/ELT → Data Warehouse
```

Prohibidas

```
Flutter → tabla operacional
Servicio A → tablas privadas de Servicio B
Data Warehouse → escritura en OLTP
Gateway → implementación de reglas de dominio
Servicio → credenciales hard-coded
DEV/QA → base de datos PROD
```

---

15. Decisiones arquitectónicas principales

ADR-001 — SOA

Decisión: organizar capacidades del negocio como servicios.
Motivo: modificabilidad, interoperabilidad y separación de responsabilidades.
Costo: mayor complejidad operacional y de observabilidad.

ADR-002 — API Gateway

Decisión: NGINX como frontera única de API.
Motivo: centralizar políticas transversales sin replicarlas en clientes.
Costo: componente crítico que debe ser redundante.

ADR-003 — Arquitectura políglota

Decisión: Java, .NET y Node.js según dominio.
Motivo: aprovechar componentes especializados y permitir evolución independiente.

Costo: más toolchains, imágenes, skills y pipelines.

ADR-004 — Propiedad de datos por dominio

Decisión: cada servicio modifica únicamente su propio modelo de persistencia.
Motivo: reducir acoplamiento.
Costo: algunas consultas requieren APIs, eventos o modelos analíticos.

ADR-005 — Data Warehouse separado

Decisión: analítica fuera de OLTP.
Motivo: evitar que reporting degrade transacciones.
Costo: consistencia analítica eventual y necesidad de pipeline de datos.

---

16. Architectural Killers y mitigaciones

Riesgo	Efecto	Mitigación
lógica de negocio en Flutter	duplicación y clientes inconsistentes	mover casos de uso a servicios
acceso directo indiscriminado a Supabase	acoplamiento y bypass de controles	APIs, RLS y encapsulación
base compartida sin ownership	monolito distribuido	propiedad por dominio
cadena síncrona extensa	fallos en cascada	eventos, timeout y circuit breaker
exceso de servicios pequeños	complejidad sin beneficio	límites por capacidad de negocio
tecnologías sin justificación	costo de operación	catálogo tecnológico aprobado
falta de idempotencia	pagos/asignaciones duplicadas	idempotency keys y constraints
sin observabilidad	incidentes difíciles de diagnosticar	métricas, logs, trazas y correlation ID
migraciones destructivas	rollback inviable	expand/contract y compatibilidad temporal
analítica sobre OLTP	degradación del flujo operativo	Data Warehouse desacoplado

---

17. Trazabilidad arquitectura ↔ calidad

Decisión	Atributos favorecidos
SOA	mantenibilidad, flexibilidad, compatibilidad
API Gateway	seguridad, interoperabilidad, fiabilidad
separación por dominio	modularidad, modificabilidad, integridad
Kubernetes	disponibilidad, escalabilidad, recuperabilidad
Docker	instalabilidad, adaptabilidad, reproducibilidad
CI/CD	testabilidad, modificabilidad, seguridad
observabilidad	analizabilidad, fiabilidad
Strategy/Factory	modificabilidad, modularidad
Adapter	reemplazabilidad, interoperabilidad
Circuit Breaker	tolerancia a fallos
Data Warehouse separado	desempeño, escalabilidad y mantenibilidad

---

18. Relación con el diseño de datos

Este SDD define **quién es responsable de los datos y cómo se accede a ellos**, pero no sustituye su especificación.

Consultar ModeloDatos.md para:

- modelo conceptual;
- modelo lógico;
- DER;
- modelo físico;
- DDL;
- diccionario de datos;
- separación de esquemas;
- integridad y restricciones;
- Data Warehouse;
- modelo estrella;
- dimensiones y hechos;
- CDC/ELT y frescura analítica.

---

19. Criterio de aceptación arquitectónica

Una versión puede promoverse a producción únicamente si:

1. cumple los Quality Gates de seguridad y mantenibilidad;
2. contract tests e integración están en verde;
3. no introduce accesos cruzados de datos no autorizados;
4. mantiene los umbrales P1 definidos en este SDD;
5. las migraciones tienen estrategia de compatibilidad y rollback;
6. existe telemetría para las operaciones críticas;
7. el artefacto desplegado es el mismo que fue validado en los ambientes previos.

---

20. Referencias

- ISO/IEC 25010:2023, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*.
- C4 Model, vistas de contexto, contenedores, componentes y código. Modelo del proyecto en `workspace.dsl`.
- OpenAPI para contratos HTTP.
- OWASP para controles de seguridad de aplicaciones y APIs.